

A Farkas certificate for a failed discharging argument

(author list to be completed)

August 31, 2026

Abstract

A discharging argument is parameterised by the amounts of charge its rules move. Fix the rule shapes and the pool of reducible configurations; then the set of amount vectors under which the argument closes is a *polyhedron*, because each local structure that the pool fails to block contributes one linear inequality in the amounts. Failure of the argument is thus infeasibility of a linear program, and infeasibility carries a Farkas certificate: a short nonnegative combination of the inequalities that sums to a contradiction, independent of any solver and checkable in exact rational arithmetic.

We give the certificate format, a solver-free exact-rational checker, and the first instance. Fix the 84 discharging rule shapes of the 2026 near-linear-time proof of the Four Colour Theorem [9] and restrict its published 8,200-configuration D-reducible pool to the 5,895 members of ring size at most 14. A 12-row irreducible certificate then shows that no nonnegative amount vector $x \in \mathbb{R}_{\geq 0}^{84}$ satisfies the local charge conditions that schema imposes over that pool: the argument cannot be made to close there by re-tuning its amounts. We say “certify” rather than “prove” in a sense made precise in §1.3 — the certificate is exact and every row is re-derived from its source by the checker, but the row set is necessary only under hypotheses that we state inside the theorem. The contradiction is legible: it lives entirely at the degree-5/degree-7 interface, and 65 of the 84 rule shapes carry a zero coefficient in every row of it. A positive control — the same construction over the full pool — is feasible, with the published amounts satisfying every generated row.

The result bears on a question Steinberger [14] posed in 2009 and explicitly declined to answer, whether an unavoidable set of D-reducible configurations of ring size at most 14 exists; we settle it for this pool and these rule shapes, and no further. Automated-discharging frameworks [15, 4, 10] solve the same kind of linear program inside a loop and treat infeasibility as a signal to enlarge the configuration set; the certificate their loops discard is exactly the object we report.

Two secondary results are included. We determine $f(r)$, the minimum number of interior vertices of a D-reducible configuration of ring size r , for $r = 8, \dots, 13$: $f = 5, 5, 6, 7, 7, 8$, exhaustively over 13.2 million generated near-triangulations and tight at every ring. And we report an independent re-verification of the published 4CT configuration corpus, which located 430 configurations of the 2026 pool that no stage of its pipeline uses and a counting defect in the 1995 `reduce.c` reference oracle.

1 Introduction

A discharging argument has two separable parts. There are the *rule shapes* — the local patterns along which charge may move — and there are the *amounts* those rules move. Almost all of the literature’s attention, and all of the difficulty of designing such an argument by hand, is in the shapes. But once the shapes and the pool of reducible configurations are fixed, the

amounts are just numbers, and the condition “every vertex ends with non-positive charge” is, at each local structure the pool fails to block, a single linear inequality in those numbers. The set of amount vectors that make the argument close is therefore a polyhedron.

This observation is not new; it is the engine of the automated-discharging frameworks of Stolee [15], Bousquet, Deschamps, de Meyer and Pierron [4] and La and Valicov [10]. What is new is what we do with it when the polyhedron is *empty*. Farkas’ lemma [7] says that an infeasible linear system admits a certificate: a nonnegative combination of its inequalities that sums to a contradiction. Applied here, that certificate is a short, human-readable object which says *precisely which local structures, in what proportions, make the schema impossible* — and it is checkable in exact rational arithmetic by a program that needs no solver, no floating point, and no trust in the enumeration that generated the other rows.

1.1 The result

The Four Colour Theorem has been proved four times by machine [1, 2, 11, 14, 9] and once in a proof assistant [8]. In every case the proof pairs an unavoidable set of reducible configurations with a discharging argument certifying unavoidability, and in every case that argument arrives with its amounts already chosen and no account of why those amounts and not others.

We give the certificate format, an exact-rational checker for it, and the first instance. The instance concerns the 2026 near-linear-time proof of the Four Colour Theorem [9], whose discharging schema has 84 rule shapes and whose pool has 8,200 D-reducible configurations of ring size up to 18. Restrict that pool to its 5,895 members of ring size at most 14 and call the result P_{14} . Theorem 4.1 then says: *no* nonnegative amount vector $x \in \mathbb{R}_{\geq 0}^{84}$ satisfies the local charge conditions the schema imposes over P_{14} . The argument cannot be rescued there by re-tuning its amounts, and the reason is twelve rows long.

The certificate is legible in a way that an LP verdict is not. Its twelve rows are nine cartwheels with a degree-7 hub and three degree-5 wheels; no degree-8, 9 or 10 structure appears at all, even though the empirical failure at the published amounts is spread across hub degrees 7 to 10. Every one of the nine degree-7 hubs has a degree-5 spoke. Only 19 of the 84 rule shapes carry a nonzero coefficient anywhere in it, so 65 of the shapes are irrelevant to the obstruction. And the contradiction reads as arithmetic: the degree-5 rows force certain amounts up, the degree-7 rows force the same amounts down, and once P_{14} stops blocking four particular cartwheels the two demands are incompatible, by exactly $10 \cdot 460 - 10 \cdot 462 = -20$.

This bears on a question posed in 2009. Steinberger [14] gave the first unavoidable set of D-reducible configurations, at the cost of ring size up to 16, and wrote that he “was not able to find an unavoidable set using only configurations of ring-size 14 or less, but [had] no opinion as to whether such a set exists or not”. We settle a sharply scoped version: for the 2026 rule shapes and the 2026 pool, ring ≤ 14 is impossible, and the twelve rows say why. We do not settle Steinberger’s question, and §4.6 states exactly how far short we fall.

1.2 Why the result type is new

Discharging has been automated in essentially this way three times. Stolee’s ADAGE [15] builds a linear program whose variables are discharging amounts and whose constraints are configurations, and reports the optimum as a density bound. Bousquet et al. [4] build the same kind of program for square colouring of planar graphs and wrap it in a loop: solve, and if the optimum is insufficient, enlarge the configuration set and solve again. La and Valicov [10]

automate the verification side of the same procedure. Cranston and West’s survey [6] gives the hand version of the method these frameworks mechanise.

In all three frameworks, an insufficient LP is a signal to change the input. Bousquet et al. enlarge $\mathcal{C}$; Stolee tries a different named rule set. The dual object that says *why* the present input cannot succeed is computed by every simplex solve — it is the Farkas ray the solver returns alongside its infeasibility verdict — and it is reported by none of them. That object is a theorem, not a diagnostic. It is finite, it is small, it names its own hypotheses, and it can be checked independently of the search that found it. Reporting it converts a failed search into a statement about the schema.

The analogy with unsatisfiability proofs. The practice we are advocating is standard one field over. A SAT solver reporting UNSAT is now expected to emit a machine-checkable proof, and large computer-assisted results are accepted on the strength of those proofs rather than of the solver: the determination of the packing chromatic number of the infinite square grid [16] rests on a 34-terabyte compressed DRAT proof, and the empty hexagon number [17] on a reduction verified in Lean. A Farkas certificate is the linear-programming analogue of an unsatisfiability proof, and in this setting it is very much smaller — twelve rows, checkable by hand in principle. The discharging literature has not adopted the practice.

The difference in setting matters too: those frameworks run their loops on schemas being designed, where enlarging the configuration set is the natural move on failure, whereas we run ours on a *published, complete* schema whose configuration set is the object under study and cannot be enlarged without changing the claim.

1.3 What “certified” means here, and what we do not claim

We use two grades and label results with them. **Proven** means a mathematical argument, in some cases with a finite exhaustive case analysis whose completeness argument is given; Theorem 6.1 is of this kind. **Certified** means a finite computation whose output is an independently checkable certificate — here, a Farkas certificate verified in exact rationals by a program that re-derives every row from its source rather than trusting the certificate’s own numbers. Theorem 4.1 is certified, not proven, and we say “certify” rather than “prove” throughout for it.

The distinction is not decorative, because three hypotheses are load-bearing and are stated inside the theorem or in §4.6. (i) The amounts are assumed nonnegative; this is used twice, and it is exactly load-bearing rather than decorative, since the monotonicity argument fails on measured examples once negative amounts are permitted. (ii) Rows are generated for every local structure the pool fails to *block*, which is what the released code does but is not the same as “every local structure that is realizable”; an unblocked but geometrically impossible structure would contribute a spurious row, which is anti-conservative for an infeasibility verdict. (iii) Three of the twelve rows come from our reconstruction of the degree-5/6 half of the discharge, which [9] discharges in a single sentence by citation. §3.2 gives that reconstruction, its line-by-line audit against the pinned source, and an exhaustive refinement check; but it remains a reconstruction, and without it the whole system is feasible.

Finally, the result is scoped to *these* 84 rule shapes and *this* pool. Other D-reducible configurations of ring ≤ 14 exist outside the published pool, and adding or replacing rule shapes is not covered.

1.4 Secondary results, and what is reported elsewhere

Two further results are included because they are short and because the second is what licenses the first. §6 determines $f(r)$, the minimum number of interior vertices of a D-reducible configuration of ring size r , for $r = 8, \dots, 13$; this is a clean exhaustive theorem, and the first quantitative refinement we know of to Birkhoff’s 1913 ring-size bound [3]. §5 reports an independent re-verification of the published 4CT corpus by a reducibility checker written from the RSST mathematics rather than ported from its code, including a 5,692,937-wheel differential against the released C++, 430 configurations of the 2026 pool that no stage of its pipeline uses, and a counting defect in the 1995 `reduce.c` reference oracle. The broader motivation is the one Tao [18] articulates for machine-assisted proof: a machine proof becomes mathematics when it yields transferable structure. A machine-learning line ran alongside this work and is reported separately.

2 Preliminaries

We keep this section to what the certificate needs and refer to [11, 13] for everything standard.

Configurations and reducibility. A *configuration* K of ring size r with k interior vertices is given by its *free completion* $S(K)$: a simple plane near-triangulation of the closed disk on $n = r + k$ vertices whose outer face is bounded by an induced cycle R of length r (the *ring*) with every ring vertex of degree ≥ 3 , whose interior is nonempty and connected with every interior vertex of degree ≥ 5 , and all of whose bounded faces are triangles. RSST’s reference parser `ReadConf` in `reduce.c` records three further conditions on any file it accepts, one of them redundant; the substantive one is that each interior vertex meets the ring in at most two separate contiguous arcs. Let $\Phi(K)$ be the set of 3-colourings of $E(R)$, up to the S_3 action on colours, that extend to a tri-colouring of $S(K)$, and write $a(K) = |\Phi(K)|$. Then K is *D-reducible* if iterating the Kempe-closure operator from $\Phi(K)$ reaches $\emptyset$, and *C-reducible* if the same holds after contracting a *contract* X of edges. The published catalogs record a nonempty X exactly for those configurations that are not already D-reducible; reducibility in the C-or-D sense is therefore strictly weaker than D-reducibility. The 2026 pool [9] and Steinberger’s set [14] are D-only; the RSST 633 are not.

Discharging schemas. A discharging argument assigns an *initial charge* $T_0(v)$ to each vertex of a hypothetical minimum counterexample, moves charge along edges by a finite set of local *rules*, and derives a contradiction from the fact that total charge is positive while every vertex ends non-positive. RSST [12] organise the local analysis around *axles*; Steinberger [14] and [9] use *cartwheels* — a hub together with its first and second neighbourhoods, with degrees carried as intervals rather than exact values, refined by branching until each is determined. A configuration in the pool *blocks* a cartwheel if it appears inside it; blocked cartwheels are discarded, since a minimum counterexample contains no reducible configuration.

The 2026 schema, which is the one we analyse, sets $T_0(v) := 10(6 - d(v))$, so that $\sum_v T_0(v) = 120$ by Euler’s formula ([9], source line 602 and Lemma `lem:120`). Each rule R is a planar degree-interval pattern with a distinguished dart; it sends an *amount* $r(R)$ of charge from a source vertex $s(R)$ to a target vertex $t(R)$ whenever the pattern matches. The

final charge is

$$T(u) = T_0(u) + \sum_{v \sim u} (\phi(v, u) - \phi(u, v)), \quad (1)$$

where $\phi(v, u)$ is the total amount sent from v to u . The published schema fixes $r(R) \in \{1, 2\}$ for all 84 rules.

Remark 2.1 (43 figures, 84 rules). [9] draws 43 rules; each is counted twice, once per reflection, except two that are reflection-symmetric, giving $43 \cdot 2 - 2 = 84$ (source line 635: “Figure `fig:rules` shows 84 rules since precisely two, the first and fourth last, are symmetric under the reflection”). This matches the 84 files in `third_party/discharging-rules/R` one for one. The paper also notes that its 12th and 13th rules are a single rule in Steinberger’s set of 42. Our amount vector ranges over all 84 coordinates independently, which is *weaker* than requiring reflection pairs to share an amount — the safe direction for an infeasibility verdict.

Survivors are permitted. This is the point on which a naive encoding of “the discharging argument closes” goes wrong, so we state it early. The charge argument need not kill every unblocked local structure: the 2026 proof leaves $5,439 + 6,790 + 3,285 + 626 + 8 = 16,148$ stage-one wheels alive on purpose, and three gluing lemmas dispose of the survivors. The correct local condition is $T \leq 0$, not $T < 0$. §4.5 reports the negative control that makes the difference concrete: the encoding demanding strict negativity is infeasible even over the *complete* 8,200-configuration pool, i.e. it would “disprove” the published theorem.

3 The amount polyhedron and its failure certificate

3.1 Amounts as variables

Fix a discharging schema: an initial charge T_0 , a finite list of rule shapes $R_1, \dots, R_N$, and a pool U of reducible configurations. The schema’s *amounts* are the numbers $r(R_1), \dots, r(R_N)$; collect them into a vector $x \in \mathbb{R}_{\geq 0}^N$. Everything else — which patterns exist, which cartwheels the pool blocks, how the refinement branches — is fixed. The question this paper asks is what the argument can do as x varies.

The basic observation is that the final charge is affine in x . Let L be a local structure in which every degree is determined, so that for each rule shape R_k it is decided which darts at the hub R_k fires along. Writing $\text{IN}_k(L)$ for the number of darts along which R_k sends charge *into* the hub and $\text{OUT}_k(L)$ for the number along which it sends charge out, (1) becomes

$$C(L, x) = T_0(d) + \sum_{k=1}^N (\text{IN}_k(L) - \text{OUT}_k(L)) x_k, \quad (2)$$

an affine function with integer coefficients: no epigraph variables, no binaries, no case split. The requirement that the hub of L ends non-positive is one linear inequality in x .

Consequently the set of amount vectors under which the argument closes over U is an intersection of half-spaces with the nonnegative orthant:

$$X(U) = \{ x \in \mathbb{R}_{\geq 0}^N : Ax \leq b \},$$

one row per local structure that U fails to block. *The schema closes for some choice of amounts if and only if $X(U) \neq \emptyset$.* When $X(U) = \emptyset$, Farkas’ lemma [7] supplies a certificate: a vector $y \geq 0$ with

$$(y^\top A)_k \geq 0 \quad \text{for every } k, \quad y^\top b < 0, \quad (3)$$

whence for any candidate $x \geq 0$ the chain $0 \leq (y^\top A)x = y^\top(Ax) \leq y^\top b < 0$ is a contradiction. The certificate is a list of (row, multiplier) pairs. It is finite, it is checkable by hand arithmetic in principle and by an exact-rational program in practice, and — this is the point of the format — it names the small set of local structures actually responsible. Nothing about it depends on the solver that found it, or on floating-point arithmetic, or on trusting the enumeration that produced the other rows.

3.2 Where the rows come from

Two families of rows are needed, and the second is not optional.

High-degree hubs. The released 2026 implementation enumerates hubs of degree 7 to 11 and refines each cartwheel until every relevant degree is determined. On a fully refined leaf L all d spokes are fixed, the per-spoke maximisation inside `amount_of_possible_charge_send` never runs, and the spoke combined rules are determined by the leaf graph, so (2) applies verbatim. The invariant the code enforces at such a leaf yields the necessary condition

$$C(L, x) \leq 0, \quad \text{and strictly } C(L, x) < 0 \text{ when either } d(L) \in \{9, 10, 11\}$$

$$\text{or } d(L) \in \{7, 8\} \text{ with no spoke of degree } \geq 7,$$

the strict cases being those in which the leaf must be pruned before an assertion is reached rather than surviving to the gluing lemmas. Note that $C = 0$ is *not* imposed: a leaf with $C < 0$ is pruned earlier, so $C \leq 0$ is exactly the necessary-and-sufficient charge condition, weaker than what the code is observed to satisfy at the published amounts. As a differential check on (2), all 10,094 published full-pool bad cartwheels evaluate to exactly 0 under our independent implementation of the row semantics, and all 10,094 satisfy $d \in \{7, 8\}$ with a spoke of degree ≥ 7 : the row semantics reproduce all three C++ assertions on the published output with 0 discrepancies.

Low-degree hubs. Taken alone, the leaf rows are satisfied by $x = 0$: send nothing, and every hub of degree ≥ 7 keeps its negative initial charge $10(6 - d)$. Any encoding that stops there is vacuous, and we measured exactly that (Table 1). The missing constraint is at $d \in \{5, 6\}$, where $T_0 > 0$ and charge must actually leave. The 2026 paper does not verify that case: it asserts the stronger $T(v) = 0$ and discharges the obligation in one sentence by citation (source line 1144), and the cited computation [14] was carried out at *Steinberger’s* amounts. Once x is re-tuned that citation no longer applies as proved, so we impose the necessary condition directly. For each hub degree $d \in \{5, 6\}$ and each spoke-degree necklace w that the pool does not block we add the row

$$\text{LB}(w, x) = 10(6 - d) - \sum_i \text{POSS}_i(x) + \sum_j \text{ALW}_j(x) \leq T(v) \leq 0, \quad (4)$$

where POSS_i over-counts sends (it sums x_k over base rules not `never_apply` out of spoke i) and ALW_j under-counts receipts (base rules that `always_apply` into the hub from spoke j). Both directions are conservative, so LB is a genuine lower bound on the hub’s final charge, valid for any $x \geq 0$ at any level of refinement.

Two things make these rows trustworthy rather than a reconstruction we merely hope is faithful. First, a quote-by-quote audit of (4) against the pinned L^AT_EX source of [9]: the initial charge, the total 120, the definition (1), and the requirement at $d \in \{5, 6\}$ (which the paper states as $T(v) = 0$, stronger than our $T(v) \leq 0$) each match a located line, and rule $R(1)$ decoded from the paper’s own figure source is a single edge with source of degree exactly 5 and amount 2, matching `rule001.rule` byte for byte — so the row $10 - 5x_{001} \leq 0$ is forced. Second, a sanity anchor: at the published amounts x_0 the maximum of LB over all 3,046 unblocked low-degree wheels is exactly 0 and never positive. Had we got this side wrong in the loose direction, x_0 would have violated it.

The coarse-refinement gap. The low-degree rows are built on the coarse wheel (hub plus spoke-degree necklace, second neighbourhood left open), and blocking is tested there. “Refining only strengthens the bound” does not cover the danger, because refining can also *delete* the row: if a coarse wheel is unblocked but every refinement of it is blocked, no vertex of a minimum counterexample carries that necklace and the row is spurious — which is anti-conservative for an infeasibility verdict. So it was checked twice. Exact model counting over the full refinement space of the certificate’s degree-5 necklaces leaves between 35.0% and 61.2% of all refinements unblocked in every case: not a knife edge. Independently, the gap closes uniformly for all 86 distinct low-degree rows at once, because `PseudoConfiguration::representative_degree` collapses any degree upper bound above 8 to the single value 9, so the coarse blocking test *is* the blocking test of the tail-maximised refinement; all 580 unblocked $d = 5$ and all 2,466 unblocked $d = 6$ necklaces have an unblocked tail-maximised refinement, with 0 exceptions of 3,046.

3.3 Why rows observed at one amount vector constrain every amount vector

The refinement forest is explored with pruning, so which leaves are observed depends on the amounts used to explore. The rows are nevertheless valid for all $x \geq 0$.

Lemma 3.1 (Monotonicity). *For every leaf L observed during the x_0 -pruned search and every $x \geq 0$, the stated condition $C(L, x) \leq 0$ (resp. < 0) is necessary.*

Proof. (a) The branching operations `update_degree_by_rule`, `concrete_degree_except_tail`, `refine_always` and `refine_never` only narrow degree intervals and never change rotations, so the shape of the forest does not depend on x . It does depend on the pool, because `fix_in_rules` branches over the non-blocked combined-rule set; but that dependence moves in the safe direction, since a smaller pool blocks fewer combined rules. We verified the containment concretely: the full-pool combined rules are a content-exact subset of the P_{14} ones ($671 \subset 681$ files), so the full-pool forest is a sub-forest of the P_{14} forest. (b) `blocked_by_reducible_configuration` is independent of x and monotone in the pool, and `prune_by_non_associated_rule` is independent of both. (c) `upper_bound_of_charge` is non-increasing along refinement whenever $x \geq 0$: narrowing a degree interval can only add `always_apply` matches, which raises the out-charge, and can only remove candidates surviving

never_apply. This is the step that uses nonnegativity, and it is the step whose faithfulness to the source is argued rather than derived; it was additionally measured on 2,145 parent/child pairs, where it holds with $x \geq 0$ and fails on 8 pairs once negative amounts are allowed. (d) Hence for any $x \geq 0$, either every ancestor of L survives, in which case the assertion at L demands $C(L, x) \leq 0$; or some ancestor A has $C(A, x) < 0$, and then $C(L, x) \leq C(A, x) < 0$ by (c). $\square$

Since $P_{14} \subseteq P_{\text{full}}$ (verified: 0 of the 5,895 names absent from the 8,200), every full-pool leaf row is also valid for the ring- ≤ 14 problem, which is what lets a certificate mix rows of both origins.

Remark 3.2 (Direction of the relaxation). The row set omits the three gluing lemmas and omits every leaf the pruned search never reached. Both omissions make the encoded feasible set *larger* than the true one. An INFEASIBLE verdict therefore transfers to the true problem, while FEASIBLE is only a non-refutation. This asymmetry is why the certificate, and not the LP verdict, is the reportable object.

3.4 Code-necessary versus math-necessary

One objection nearly killed the result and is generic to this method. The released code counts only **always_apply** rules outward, and stops refining once no rule is **dominantly_apply-but-not-always_apply**; since **dominantly_apply** is strictly stronger than **has_intersection**, a leaf can retain rules that are neither **always_apply** nor **never_apply** on an outgoing dart — rules that might fire in some realization but are not counted in the out-charge. Measured on the certificate’s own leaves, 11 of 5,292 (rule, out-dart) pairs are undetermined in this sense, and for those leaves C strictly *over*-estimates the hub’s true final charge, so $C(L, x) \leq 0$ would be a condition of the released *verification procedure* rather than of the mathematics. Two blunt repairs — dropping such rows, or weakening them — both restore feasibility (Table 1), so the distinction is not hypothetical.

The repair we use instead is to pass to the *tail-maximised* leaf, narrowing every open tail $[a, 9]$ with $a < 9$ to $[9, 9]$; since 9 abbreviates “9 or more”, this restricts attention to the sub-family of realizations in which every unpinned second neighbour has large degree. If the tail-maximised leaf (i) has a fully determined outgoing side, (ii) yields the identical row — same coefficients, same constant, same strictness — and (iii) is still unblocked by the pool, then in those realizations the out-charge is exact while the in-charge can only exceed the branch’s flag sum, so the true final charge is at least $C(L, x)$ and $T \leq 0$ forces $C(L, x) \leq 0$. Undetermined incoming rules are harmless precisely because they push the true charge up. All twelve rows of the certificate below pass this audit: the nine degree-7 rows satisfy (i)–(iii) individually, including the five whose original leaves had undetermined outgoing pairs, and the three degree-5 rows are mathematically necessary by construction, being already in lower-bound form (`results/steinberger/iis_math_necessity.json`).

4 The instance: the ring- ≤ 14 sub-pool of the 2026 schema

4.1 The question, and an empirical precursor

Steinberger’s unavoidable set [14] is D-only but reaches ring size 16; RSST’s 633 stay at ring ≤ 14 but use contracts. Steinberger raises the obvious question and declines to take a position

(third_party/arxiv-0905.0043/src/4c.tex, line 137):

“Appel and Haken used only configurations of ring-size at most 14. . . The unavoidable set of Robertson et al. also has only configurations of ring-size 14 or less. Our unavoidable set uses configurations of ring-size up to 16. . . **We were not able to find an unavoidable set using only configurations of ring-size 14 or less, but we have no opinion as to whether such a set exists or not.**”

The 2026 pool [9] is D-only, has 8,200 configurations of ring size up to 18, and contains 5,895 of ring size ≤ 14 ; call that sub-pool P_{14} .

Before any linear programming we ran the released pipeline with the pool replaced by P_{14} , the 84 rule shapes and the published amounts x_0 left untouched. It fails, and in a structured way: 681 non-blocked combined rules rather than the baseline 671 (ten revived, all ten predicted in advance by a gap analysis with 0 unexpected re-blockings), a per-wheel job count of 16,157 rather than 16,148, and then 833 of those jobs aborting — 780 on `assert(C == 0)` (530 at $d = 7$, 250 at $d = 8$) and 53 on `assert(d == 7 || d == 8)` (51 at $d = 9$, 2 at $d = 10$), the two failure kinds perfectly stratified by hub degree.

That is a statement about x_0 alone, and the obvious response is to re-tune the amounts. The theorem below says no re-tuning can work. The 833 aborted jobs are also the raw material for it: re-run against an NDEBUG build with the assertions compiled out, they yield 77,185 additional fully refined leaves that the assert-guarded run never recorded.

4.2 The theorem

Theorem 4.1 (Certified; see §1.3 for the grade and §4.6 for the hypotheses). *Fix the 84 discharging rule shapes of [9] (third_party/discharging-rules/R), and let P_{14} be the sub-pool of the published 8,200-configuration D-reducible pool consisting of its 5,895 members of ring size at most 14. Then there is no vector of nonnegative rule amounts $x \in \mathbb{R}_{\geq 0}^{84}$ satisfying the local charge conditions that the 2026 schema imposes over P_{14} : that is, no $x \geq 0$ under which every fully refined cartwheel leaf L that P_{14} fails to block satisfies $C(L, x) \leq 0$ — strictly $C(L, x) < 0$ in the cases listed in §3.2 — and every unblocked degree-5 or degree-6 wheel w satisfies $LB(w, x) \leq 0$. In particular the discharging argument of [9] cannot be made to close over P_{14} by re-tuning its amounts.*

All twelve rows of the certificate are non-strict, so infeasibility holds over the nonnegative reals, hence a fortiori over $\mathbb{Z}_{\geq 0}^{84}$ and over the box $\{1, 2\}^{84}$ that [9] allows itself.

The system behind the theorem has 48,509 distinct rows: 97,860 fully refined cartwheels yield 48,423 distinct leaf rows after deduplication by (coefficients, constant, strictness), to which the 86 distinct low-degree rows are added. The leaves are 10,094 published full-pool bad cartwheels, 10,581 from the 15,324 per-wheel jobs that completed, and the 77,185 recovered from the 833 jobs that aborted. By hub degree the rows are 40,257 at $d = 7$, 8,070 at $d = 8$, 93 at $d = 9$, 3 at $d = 10$, 26 at $d = 5$ and 60 at $d = 6$; the 93 and 3 are exactly the strict rows corresponding to the 51 + 2 degree-range failures above, and the 3,046 unblocked low-degree wheels collapse to only 86 distinct rows.

4.3 The certificate

A solver returns a 19-row Farkas support with $y^\top b = -30$. Greedy deletion filtering shrinks it to the 12-row *irreducible infeasible subsystem* of Table 2: deleting any single one of the

Table 1: The four headline variants and the two lossy diagnostics of §3.4. “ x_0 viol.” counts rows the published amounts violate. Rows are deduplicated by (coefficients, constant, strictness). Full table of eight variants, and reproduction instructions, in the artifact repository (see the availability statement).

pool	low-degree rows	rows	x_0 viol.	status
full 8,200 (control)	included	3,511	0	feasible
full 8,200 (control)	omitted	3,425	0	feasible (vacuously, $x = 0$)
P_{14} (main)	included	48,509	9,647	infeasible
P_{14} (main)	omitted	48,423	9,647	feasible (vacuously, $x = 0$)
P_{14} , rows dropped	included	42,121	8,699	feasible — too lossy
P_{14} , rows weakened	included	52,400	8,741	feasible — too lossy

Table 2: The 12-row irreducible certificate for Theorem 4.1, with multipliers y and $y^\top b = -20$. “rhs” is the row’s right-hand side and $C(x_0)$ the leaf charge at the published amounts, so a positive entry marks a row x_0 violates. Every row is non-strict. The full 84-column coefficient vectors, the 19-row support, and the machine-readable certificates are in the artifact repository (see the availability statement).

y_i	d	spoke degrees	rhs	$C(x_0)$	source	origin
75	7	5,5,8,5,6,5,7	10	0	d7_1155_1	full $\cap P_{14}$
75	7	5,5,5,8,5,5,7	10	0	d7_166_1	full $\cap P_{14}$
25	7	5,5,7,5,5,7,7	10	0	d7_710_10	full $\cap P_{14}$
75	7	5,5,7,5,6,5,8	10	0	d7_722_3	full $\cap P_{14}$
55	7	5,5,7,5,7,5,7	10	0	d7_740_19	full $\cap P_{14}$
60	7	5,7,5,7,6,7,7	10	2	d7_3358_2133	P_{14} only
15	7	5,7,5,7,6,7,7	10	2	d7_3358_2134	P_{14} only
55	7	5,7,5,7,7,6,7	10	2	d7_3372_1881	P_{14} only
25	7	5,7,5,7,7,7,7	10	3	d7_3376_1132	P_{14} only
75	5	5,5,7,6,7	−10	−2	d5-55767	low-degree
168	5	5,7,5,7,7	−10	0	d5-57577	low-degree
219	5	5,7,6,7,7	−10	0	d5-57677	low-degree

twelve restores feasibility, verified for all twelve. This is the smallest human-inspectable object carrying the whole result. Its arithmetic fits on one line: the nine degree-7 rows have right-hand side 10 and multipliers summing to 460, the three degree-5 rows have right-hand side −10 and multipliers summing to 462, so

$$y^\top b = 10 \cdot 460 - 10 \cdot 462 = -20 < 0,$$

while $(y^\top A)_k \geq 0$ holds in all 84 columns — in fact 78 of the 84 columns cancel exactly and only six carry slack, which is why the contradiction is tight.

What it says. The certificate is legible in a way LP output usually is not.

- **No degree-8, 9 or 10 row appears.** The contradiction needs none of the 53 degree-range failures nor any of the 250 degree-8 failures seen empirically. The irreducible cause sits entirely at hub degree 7.

- **Every one of the nine degree-7 hubs has a degree-5 spoke**, so the whole obstruction lives at the degree-5/degree-7 interface.
- **Only 19 of the 84 rule shapes carry a nonzero coefficient anywhere in the twelve rows** — rule001 together with the reflection pairs of rule002, 003, 004, 008, 009, 010, 011, 023 and 032. The remaining 65 shapes are irrelevant to the obstruction: re-tuning them cannot help.
- **Read as arithmetic**, the degree-5 rows say a degree-5 vertex must give away all 10 units of its charge, forcing the amounts on rule001–rule004 *up*; the degree-7 rows cap what a degree-7 hub with degree-5 neighbours may receive back, pulling the same amounts *down*. With P_{14} no longer blocking four offending degree-7 cartwheels, the two demands cannot both be met.

The infeasibility is caused by the ring- ≤ 14 rows. Delete the four P_{14} -only rows from the 19-row support and the remaining 15 — each of which is also valid for the full pool — are feasible, with x_0 satisfying all 15. The contradiction is not lurking in the full-pool constraints; it is created by exactly the cartwheels P_{14} fails to block. Independently, those four rows come from wheels d7_3358, d7_3372 and d7_3376 (d7_3358 contributes two leaves) — precisely three of the wheels whose job the released C++ aborted with `Assertion failed: (C == 0)`. Our row semantics and the C++ assertions agree wheel by wheel on the certificate’s own support.

4.4 Checking it

`tools/verify_farkas.py` is solver-free and uses exact rational arithmetic throughout; there is no floating point anywhere on the checking path. Critically, it does not trust the certificate’s stored coefficients: it re-derives every row from its source — cartwheel files re-parsed and re-run through the leaf-row constructor, low-degree tags rebuilt from the necklace and re-run through the lower-bound constructor — and fails loudly on any mismatch. It then checks $y \geq 0$, $y \neq 0$, $(y^\top A)_k \geq 0$ for all 84 columns and $y^\top b < 0$, prints the contradiction chain of (3), and independently confirms which rows x_0 violates. Deliberate corruption of a single coefficient is rejected with an exact claimed-versus-re-derived diff. A reader who accepts the row semantics of §3.2 need trust nothing else in this paper to check Theorem 4.1.

4.5 Controls

Positive control. The identical construction over the full 8,200-configuration pool is feasible, and the published amounts x_0 satisfy every one of its 3,511 rows with 0 violations (Table 1). So the encoding is not vacuously infeasible: it distinguishes the two pools, and it accepts the published proof.

Negative control. The encoding a naive reading suggests — *every* unblocked stage-one wheel must have strictly negative charge — is not a correct rendering of “the argument closes”, because survivors are permitted (§2). We ran it anyway. It is infeasible at iteration 0 on the *full* pool, with a verified 10-row certificate ($y^\top b = -175$; three degree-5, two degree-6 and five degree-7 rows). That encoding would therefore “disprove” the published theorem, which is the concrete demonstration that it is the wrong one. We report it as a measurement and never as evidence for the headline.

Ablations and robustness. The degree-6 rows together with all leaf rows are feasible; the degree-5 rows alone are already infeasible. Six solver configurations all report infeasible, and a uniform slack of 0.082 on every right-hand side is the minimum needed to restore feasibility. The formulation was also handed to a reviewer prompted to refute it across ten named attack lines, which produced one real error (an over-strong pool-independence claim in Lemma 3.1(a)), one scope correction (§3.4), and one genuine gap, closed exactly (§3.2) — none of which changed the certificate.

4.6 Scope

1. **These 84 shapes only.** Nothing here rules out a different or larger set of rule shapes; that requires column generation over planar rule patterns (§7).
2. **This pool only.** P_{14} is exactly the ring- ≤ 14 part of the published pool. Many D-reducible configurations of ring ≤ 14 lie outside it.
3. **The $d \leq 6$ rows are a reconstruction** (§3.2) and are load-bearing: without them the system is feasible.
4. **Unblocked $\neq$ realizable.** Rows are generated for every local structure the pool fails to block, exactly as the released code decides which cartwheels to analyse. A structure that is geometrically impossible for a reason outside the pool would contribute a spurious row, which is anti-conservative for an infeasibility verdict. It is also precisely the standard the published proof uses.
5. **$x \geq 0$ is a hypothesis**, used in Lemma 3.1(c) and in (4), and it is load-bearing rather than decorative (monotonicity fails on 8 measured pairs once negative amounts are allowed). In the other direction there is nothing to hedge: the published schema fixes $r(R) \in \{1, 2\}$, and $\{1, 2\}^{84} \subset \mathbb{Z}_{\geq 0}^{84} \subset \mathbb{R}_{\geq 0}^{84}$, so deciding infeasibility over the nonnegative reals is strictly stronger than the schema’s own amount space requires.

5 The verification that licenses the certificate

None of the linear-programming argument of the preceding sections means anything unless the reducibility checker feeding it, and the row semantics built on top of that checker, are correct. Our checker `fourcolor.reduce.check` was written directly from the mathematics of [13] (the `reduce.tex` write-up, in particular its expansion of [11, Theorem (3.2)]), not ported from `reduce.c`. Agreement with the released C programs is therefore genuine differential evidence, not a tautology: two independent readings of the same theorem, implemented by different people at different times in different languages, are being asked whether they compute the same numbers. Table 3 summarizes the five legs of that comparison; full logs, configuration files, and rerun scripts are in the artifact repository (see the availability statement).

Why the differential matters. The 5,692,937-wheel exact match is what licenses treating the leaf charge as an affine function of the amount vector: it is the bridge between our Python row semantics and the C++ that the published proof actually runs, so the LP rows we hand the solver are demonstrably the rows the proof means.

Table 3: The five verification legs. “Outcome” reports what was certified computationally, not proved mathematically. Row 3 has no published $|\Phi(K)|$ to compare against – the released 2026 .conf files carry only an “ n r ” header – so “confirmed” there means only that our checker independently certifies each configuration D-reducible, not that it matches a prior count.

What	Scale	Outcome
Catalog re-verification, RSST 633 ([13] unavoidable.conf)	633 configurations, rings 6–14	Computed $ \Phi(K) $ and consistent match published header fields for all 633; 249 D-reducible, 384 C-reducible via a nonempty published contract and not D-reducible.
Catalog re-verification, Steinberger $\mathcal{U}_{2822}$	2,822 configurations, rings 6–16, all header $b = 0$	2,822/2,822 agree with published headers, 0 disagreements.
2026 pool	7,697 configurations	Independently confirmed D-reducible, 0 anomalies, 12.3 CPU-hours. Honest accounting: 8,200 = 7,697 verified + 430 unused-and-removed + 73 unverified (rings 17–18, outside our engines’ ring budget).
Wheel-level differential, Python reimplement vs. released C++	5,692,937 wheel candidates, hub degrees 7–11 (11,165 + 48,915 + 217,045 + 976,887 + 4,438,925)	Every degree reports <code>exact_match: true</code> : 0 false positives, 0 false negatives.
End-to-end reproduction of released C++ pipeline	8,200-configuration pool (19,754 after symmetry expansion), 84 rule shapes, Debug build with all assertions live, wall 4h38m	Every count matches the published proof: 1,832 combined rules, 671 non-blocked; bad wheels 5,439/6,790/3,285/626/8 at $d=7, \dots, 11$; 10,094 bad cartwheels (9,366 at $d=7$, 728 at $d=8$); three gluing lemmas all passed (5,365 / 1,618 / 298 cartwheels).

5.1 Two defects found in released code

A legacy oracle that aborts. `build/reduce_stein`, compiled from the Steinberger-extended `reduce.c` (needed because the unmodified 1995 code has `MAXRING = 14`), processes 783 of Steinberger’s 2,822 configurations and then aborts with `SIGABRT` (exit 134), truncating mid-number. The crashing input is preserved as `build/crash783.conf`: a ring-16 configuration with $n = 28$ and $a = 139,480$. Our checker completes it in 227.7 s, returning $a = 139,480$, matching the published header. The binary corroborates the first 783 and then dies; our own checker, run against the published headers, carries the remaining 2,039 unaided.

A counting defect in `reduce.c`. On 70 sampled exhaustively-generated RSST-legal configurations run through `build/reduce_rsst`, there were 0 verdict mismatches but 6 oracle-inconclusive cases: on those 6 the oracle recomputed a different $|\Phi(K)|$ than our checker supplied and aborted with `*** ERROR: DISCREPANCY IN NUMBER OF EXTENDING COLOURINGS ***` before printing a verdict. A third, from-scratch count of $|\Phi(K)|$, sharing no code path with the production checker (different edge indexing, different triangle enumeration, no gauge fixing, no greedy variable ordering), agrees with our checker in all 6 and disagrees with the oracle in all 6: (ring, n , ours = brute force, oracle) equal (8, 12, 72, 48), (9, 13, 147, 94), (10, 15, 367, 230),

(10, 15, 365, 243), (10, 15, 408, 250), and (11, 17, 995, 532). The evidence points to `reduce.c`'s `strip()` edge-numbering heuristic – a hand-optimized greedy ordering routine, documented in its own comments as chosen to shrink the search tree – as the outlier on a class of inputs (small, exhaustively generated, generic near-triangulations) that its original validation against a hand-curated catalog never exercised. None of the six configurations' D-reducibility verdict is in question: all are non-D-reducible, and the oracle never disputed the verdict, only the intermediate count. Our checker agrees with the oracle's header *a* on all 3,455 hand-curated catalog configurations (633 + 2,822) with zero mismatches, so this is not a general defect. See `results/theorem/fr_table/crosscheck.json`.

5.2 A pool 430 configurations larger than it needs to be

Coverage is sharply two-level. At the wheel level only 85 of the 8,200 configurations ever act as a blocker anywhere across $d = 7, \dots, 11$. At the cartwheel level 7,422 distinct configurations appear as blockers over 2,253,185 logged blocking events – 90.5% of the pool, and a lower bound, since the instrumentation records only the first matching configuration per event. The intuition that a small core carries the proof is right at the wheel stage and wrong at the cartwheel stage.

A first prune tracked three blocking roles and found 673 unused; removing them made the pipeline fail (all three gluing checks aborted, the bad-cartwheel count collapsed from 10,094 to 0). That failure discovered a fourth blocking role: the gluing lemmas also consult the pool (2,199 distinct glue-blockers). Tracking all four roles gives 430 configurations with zero usage anywhere. Removing exactly those and re-running the entire pipeline gives PASS with counts identical to the full-pool proof in every field.

Proposition 5.1 (Certified). *The 2026 proof goes through verbatim with a pool of 7,770 configurations rather than 8,200, a reduction of 5.2%.*

This is a floor we reached, not a claimed minimum.

We are not aware of any prior independent verification of the 2026 pool's reducibility; the released `computer-checks` repository (`third_party/GIT_PINS.txt`, commit 6cb85666) consumes the pool as an input and contains no reducibility checker of its own.

6 The minimum interior size of a D-reducible configuration

Write $f(r)$ for the minimum number of interior vertices of a D-reducible configuration of ring size r .

Theorem 6.1 (Proven, exhaustive). *For each ring size $r \in \{8, \dots, 13\}$, every D-reducible configuration of ring size r has at least $f(r)$ interior vertices, where*

$$(f(8), f(9), f(10), f(11), f(12), f(13)) = (5, 5, 6, 7, 7, 8).$$

Each bound is tight: a D-reducible configuration with exactly $f(r)$ interior vertices exists in the published catalogs.

Birkhoff [3] proved that a reducible configuration has ring size at least 6; to our knowledge Theorem 6.1 is the first published lower bound on interior size as a function of ring size.

Published catalogs give per-ring interior minima, but a catalog minimum is only an upper bound on $f(r)$ – the exhaustive negative side, that nothing smaller can possibly be D-reducible, is what is new.

6.1 The enumeration

For each r and each n from $r + 1$ to $r + f(r) - 1$ we generated every near-triangulation of the disk with outer face of size r on n vertices using `plantri -Pr -c3 -a n` (`plantri` 5.5 [5], pinned with its SHA-256 digest), filtered to the RSST configuration class, and ran the D-reducibility checker on every survivor. `plantri` is a canonical-construction-path generator producing exactly one representative per isomorphism class, which is what makes “sweep every n below threshold” an exhaustive search rather than a sample; with `-P` and no `-m`, `-c3` defaults to `-c3m3`, i.e. no chords and no outer-face vertex of degree 2.

r	interior range	raw	cand	excl	valid	D-red.
8	1–4	436	18	6	12	0
9	1–4	666	35	17	18	0
10	1–5	13,338	221	127	94	0
11	1–6	293,555	1,631	1,028	603	0
12	1–6	509,288	4,092	3,004	1,088	0
13	1–7	12,389,976	36,392	27,888	8,504	0
total		13,207,259	42,389	32,070	10,319	0

Table 4: Exhaustive sweep below the interior threshold for each ring size. “raw” is the count of near-triangulations from `plantri`; “cand” passes the interior filter (nonempty connected interior of minimum degree 5); “excl” additionally fails the RSST ring-contact-arc condition; “valid” is the true RSST configuration class. Every D-reducibility verdict is 0. `results/theorem/fr_table/manifest_r*.json`.

RSST’s reference parser `ReadConf` in `reduce.c` records three further conditions on any file it accepts – one of them redundant – and we enforce all three; the substantive one is that each interior vertex meets the ring in at most two separate contiguous arcs. This is genuine structure, not bookkeeping: it is not implied by the near-triangulation conditions, and its exclusion rate rises from 33% at $r = 8$ to 77% at $r = 13$. Excluded graphs were nevertheless retained with a D-reducibility verdict computed anyway, so a D-reducible-but-not-RSST-legal graph would have been surfaced loudly; none was – every one of the 32,070 exclusions is non-D-reducible too, so the filter never changed a verdict, only the population count. The $r = 13$ cells at interior 6 ($n = 19$) and interior 7 ($n = 20$) were run 8-way sharded by residue class, roughly 2,250s and 13,300s per shard, giving 1,445 and 6,710 valid configurations and 0 D-reducible.

6.2 Verification and tightness

Up to 20 RSST-legal below-threshold configurations per ring were serialised and run through `build/reduce_rsst`, compiled from the pinned `reduce.c`: 70 sampled, 0 verdict mismatches, 6 oracle-inconclusive. The 6 inconclusive cases trace to a defect in the oracle’s colouring count, adjudicated in our favour by a third from-scratch implementation; see §5 for the numbers.

Tightness witnesses, verified both by our checker and by the oracle, come from the published catalogs: $r = 8$: `D0003.conf` ($n = 13$, $a = 100$); $r = 9$: `D0006.conf` ($n = 14$, $a = 211$); $r = 10$: `D0015.conf` ($n = 16$, $a = 536$); $r = 11$: `D0024.conf` ($n = 18$, $a = 1,363$); $r = 12$: `D0143.conf` ($n = 19$, $a = 2,863$) – all five from the 2026 D pool [9] – and $r = 13$: `126.504366` from the RSST 633 [13] ($n = 21$, $a = 6,954$). `results/theorem/fr_table/tightness_witnesses.json`.

Theorem 6.1 is scoped to D-reducibility. The corresponding minimum over configurations that are C-reducible via a nonempty contract but not D-reducible is strictly smaller at rings 8 and 11, so the D-scoping in the statement is not cosmetic.

7 Discussion and open problems

Certificates for the automated-discharging frameworks. The immediate consequence of §3 is not about the Four Colour Theorem. Any of the frameworks of [15, 4, 10] can emit a Farkas certificate at no extra cost, because their solvers already compute one whenever a round of the loop fails; what is missing is the decision to record it, the row-provenance bookkeeping that makes it re-derivable, and an exact-rational checker. The value is highest exactly where those loops currently give the least information: when a configuration set is believed to be complete and the LP still fails, the certificate distinguishes “the rule shapes are wrong” from “this particular local structure is not covered”, and it names the structure. We would like to see one reported for a schema someone is actively designing.

Removing the fixed-shapes hypothesis. Theorem 4.1 fixes the 84 published rule shapes; the natural strengthening — “no discharging schema of bounded rule complexity closes over the ring- ≤ 14 pool” — requires column generation. Given a violated row, one must price out a new planar degree-interval pattern with a charge dart that would repair it, and that pricing subproblem is a search over planar patterns rather than a linear program. It is the main technical obstacle. A tractable intermediate is to enumerate all rule shapes below a fixed size bound and re-run the same machinery over the enlarged column set: infeasibility there would be a substantially stronger statement, and the certificate format is unchanged.

Steinberger’s question proper. Settling whether *some* D-only unavoidable set of ring size ≤ 14 exists needs both of the above plus a way to range over pools. We note only that the obstruction we found is not diffuse: it is a single interface, and 65 of the 84 shapes play no part in it. If there is a positive answer, the twelve rows say where new rules would have to act.

$f(r)$ beyond $r = 13$. The $r = 13$ interior-7 cell alone required 11,534,001 generated graphs and roughly 30 core-hours; $r = 14$ at interior 8 is about two orders of magnitude larger and is out of reach of the present pipeline on a workstation. The promising route is to prune the generation with a necessary condition on $a(K)$ rather than to run the reducibility check on every candidate. The corpus already gives the upper bounds $f(14) \leq 9$, $f(15) \leq 10$ and $f(16) \leq 10$ from catalog witnesses.

A coloring-mass law, in progress. The pruning condition just mentioned is the subject of a companion line of work, reported separately: an exact closed form $a = (2^r + 2(-1)^r)/6$ for the wheel, a proven cap $a \leq 11 \cdot 2^{r+k-7}$ tight at every wheel, a proof that the growth constant is $3/2$

at $k = 2$, and an obstruction theorem showing that no bound factoring through $a \leq P(S, 4)/24$ can reach the empirically sharp constant $4/3$. Paired with an empirical threshold law it would reproduce the whole $f(r)$ table analytically to within one interior vertex at every ring; the sharp cap is at present a conjecture and the threshold half is corpus-supported only. The current state, including the corpus and the frontier data, is in the artifact repository (see the availability statement).

Formalisation. Both results here are within reach of a proof assistant given Gonthier’s formalisation [8] of the reducibility machinery. The finite content of Theorem 6.1 is a replay of 10,319 configuration checks. Theorem 4.1 is more tractable still at the arithmetic level — twelve rows over the rationals — but the row-derivation step, from a cartwheel to its 84 coefficients, carries the real formalisation burden, and it is the step on which the theorem’s faithfulness to [9] depends.

Data and code availability

The certificate (both the 19-row support and the 12-row irreducible core, in machine-readable form with full 84-column coefficient vectors), the solver-free exact-rational checker `tools/verify_farkas.py`, the row generator `tools/lp_discharge.py`, the enumeration manifests and per-configuration records behind Theorem 6.1, the catalog re-verification reports, the pipeline reproduction logs and the pool-coverage measurements are all in the artifact repository at https://github.com/<repository>/four_color_digestion, which will be archived with a DOI on publication. All third-party inputs — the four arXiv e-prints, the `plantri` generator, the released rule and configuration files and the released C++ checker — are pinned in-repository with recorded SHA-256 digests (`third_party/CHECKSUMS.sha256`, `third_party/GIT.PINS.txt`). Following Steinberger’s precedent for the 2,822-configuration set, the bulk machine output — the 48,509 generated rows, the 97,860 refined cartwheels, the 13.2 million generated near-triangulations and their verdicts — is not reproduced here in any form; it is in the repository, with a claim-to-artifact table naming the file behind every number printed above.

References

- [1] Kenneth Appel and Wolfgang Haken. Every planar map is four colorable. Part I: Discharging. *Illinois Journal of Mathematics*, 21(3):429–490, 1977.
- [2] Kenneth Appel, Wolfgang Haken, and John Koch. Every planar map is four colorable. Part II: Reducibility. *Illinois Journal of Mathematics*, 21(3):491–567, 1977.
- [3] George D. Birkhoff. The reducibility of maps. *American Journal of Mathematics*, 35(2):115–128, 1913.
- [4] Nicolas Bousquet, Quentin Deschamps, Lucas de Meyer, and Théo Pierron. Square coloring planar graphs with automatic discharging. *SIAM Journal on Discrete Mathematics*, 38(1):504–528, 2024. arXiv:2204.05791.

- [5] Gunnar Brinkmann and Brendan D. McKay. Fast generation of planar graphs. *MATCH Communications in Mathematical and in Computer Chemistry*, 58(2):323–357, 2007. The `plantri` generator; version 5.5 (17 May 2024) pinned.
- [6] Daniel W. Cranston and Douglas B. West. An introduction to the discharging method via graph coloring. *Discrete Mathematics*, 340(4):766–793, 2017.
- [7] Julius Farkas. Über die Theorie der einfachen Ungleichungen. *Journal für die reine und angewandte Mathematik*, 124:1–27, 1902.
- [8] Georges Gonthier. Formal proof—the four-color theorem. *Notices of the American Mathematical Society*, 55(11):1382–1393, 2008.
- [9] Yuta Inoue, Ken-ichi Kawarabayashi, Atsuyuki Miyashita, Bojan Mohar, Carsten Thomassen, and Mikkel Thorup. The four color theorem with linearly many reducible configurations and near-linear time coloring, 2026. Author list taken from the `\author` block at line 111 of the pinned e-print source; archive sha256 `e517cf6f...92bb8b`.
- [10] Hoang La and Petru Valicov. Computer assisted discharging procedure on planar graphs: Application to 2-distance coloring, 2022. Preprint; no published version is recorded in DBLP.
- [11] Neil Robertson, Daniel P. Sanders, Paul Seymour, and Robin Thomas. The four-colour theorem. *Journal of Combinatorial Theory, Series B*, 70(1):2–44, 1997.
- [12] Neil Robertson, Daniel P. Sanders, Paul Seymour, and Robin Thomas. Discharging cartwheels, 2014. Ancillary files: `discharge.c`, the rule file and `present7-present11`.
- [13] Neil Robertson, Daniel P. Sanders, Paul Seymour, and Robin Thomas. Reducibility for the four-color theorem, 2014. Ancillary files: `reduce.c`, `unavoidable.conf`.
- [14] John P. Steinberger. An unavoidable set of D -reducible configurations, 2009. Published version: *Trans. Amer. Math. Soc.* 362 (2010), no. 12, 6633–6661; pinned locally as `third_party/arxiv-0905.0043`.
- [15] Derrick Stolee. Automated discharging arguments for density problems in grids, 2014. Preprint; no published version is recorded in DBLP.
- [16] Bernardo Subercaseaux and Marijn J. H. Heule. The packing chromatic number of the infinite square grid is 15. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2023), Part I*, volume 13993 of *Lecture Notes in Computer Science*, pages 389–406. Springer, 2023. arXiv:2301.09757.
- [17] Bernardo Subercaseaux, Wojciech Nawrocki, James Gallicchio, Cayden R. Codel, Mario Carneiro, and Marijn J. H. Heule. Formal verification of the empty hexagon number. In *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *LIPICs*, pages 35:1–35:19. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024. arXiv:2403.17370.
- [18] Terence Tao. Machine-assisted proof. *Notices of the American Mathematical Society*, 72(1):6–13, 2025.